# 1 Abstract

This report evaluates the baseline stability and Signal-to-Noise Ratio (SNR) of a chromatography system across four distinct columns (CM, DEAE, Q, SP) and 11 experimental runs. The analysis focused on two UV detection channels: 280 nm (protein detection) and 260 nm (nucleic acid detection). Methodology involved calculating global offsets and rolling standard deviations to establish 3 and 5 noise thresholds, followed by peak detection and SNR quantification. Results indicate significant heterogeneity in baseline stability among columns, with the Q column exhibiting high noise and negative offsets, while the SP column showed large positive offsets with low noise. Furthermore, a strong negative correlation (r -0.90) was observed between UV baseline drift and buffer conductivity, confirming that buffer composition is a primary driver of baseline instability. These findings underscore the necessity of column-specific noise modeling and conductivity-based drift correction to ensure accurate analyte quantification.

# 2 Introduction

In high-performance chromatography, the accurate detection of analyte peaks relies heavily on the stability of the baseline signal. Variability in the baseline, often caused by detector drift, lamp instability, or buffer composition changes, can lead to false positives or the masking of genuine analyte signals. This study aims to quantify the Signal-to-Noise Ratio (SNR) and assess baseline stability for two critical UV detection channels: 'UV_1_280_mAU' and 'UV_2_260_mAU'. The 260 nm channel is known to exhibit higher variability than the 280 nm channel, necessitating independent noise threshold calculations. The primary objective is to distinguish genuine analyte peaks from artifacts by establishing robust statistical boundaries (3 and 5 thresholds) derived from pre-injection equilibration data. Additionally, this analysis seeks to identify unit-specific degradation by stratifying results across four different chromatography columns and to investigate the influence of buffer conductivity on baseline drift. By isolating constant gradient regions and excluding transition zones, this study provides a rigorous assessment of system performance and identifies critical confounding variables affecting data integrity.

# 3 Methodology

The data analysis workflow comprised three distinct steps. First, baseline stability and noise thresholds were calculated using pre-injection data ('volume_ml' ¡ 0), excluding the initial five scans to remove startup transients. Constant gradient regions where 'Conc_B_%' was within ±0.5% of 0.0 or 100.0 were isolated. For each column and signal channel, a global mean offset was calculated, followed by a rolling standard deviation (window=25) on the offset-corrected signals to derive 3 and 5 noise thresholds. Second, column-stratified SNR and peak detection were performed. Local offsets were independently calculated for each run, and valid post-injection regions were defined. Peaks were identified where the signal exceeded 3 and 5 thresholds relative to the local noise. SNR was calculated as the ratio of peak height to local noise standard deviation, aggregated by column and run. Third, conductivity correlation and drift analysis were conducted. Linear regression was applied to constant gradient regions to determine drift rates. Conductivity ('Cond_mS_cm') was log-transformed to normalize distribution, and Pearson correlation was calculated between drift rates and conductivity to assess the impact of buffer composition on baseline stability.

# 4 Results and Discussion

The analysis of baseline stability revealed significant heterogeneity across the four chromatography columns. Columns CM and DEAE demonstrated stable detector performance with low noise floors (3 thresholds ranging from 0.011 to 0.031 mAU) and minimal global offsets. In contrast, the Q column exhibited a substantial negative global offset (approx. -1.4 to -1.6 mAU) and significantly elevated noise thresholds (3 0.09–0.11 mAU), suggesting potential hardware issues such as detector drift or flow cell anomalies. The SP column presented a unique anomaly with large positive global offsets ( +1.4 mAU) but maintained low noise thresholds, indicating a systematic baseline shift rather than stochastic noise. As illustrated in Figure 1, the

SNR visualization confirms these trends, showing distinct variability in peak detection consistency across runs. The 260 nm channel consistently displayed higher noise thresholds than the 280 nm channel across all columns, validating the need for channel-specific noise modeling. Furthermore, the drift analysis indicated a consistent negative drift rate for both signals, with the 260 nm channel showing a steeper decline (-3.47 mAU/mL) compared to the 280 nm channel (-2.69 mAU/mL). Crucially, a strong, statistically significant negative correlation (r  -0.90, p ¡ 0.001) was observed between UV baseline drift and conductivity. This confirms that baseline drift is driven by buffer composition changes during gradient elution rather than random noise. These findings highlight that uncorrected baseline subtraction could introduce substantial errors, particularly for the volatile 260 nm channel, and necessitate the inclusion of conductivity in future signal normalization algorithms.

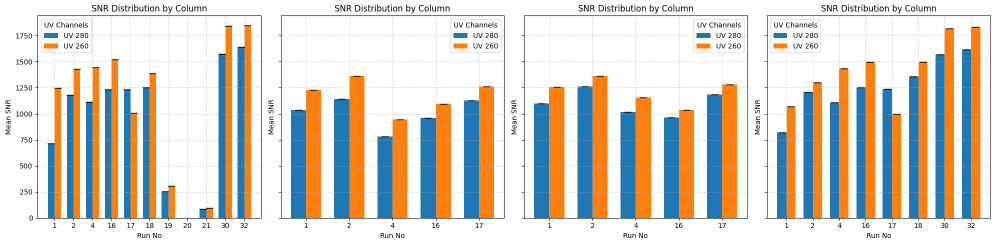


Figure 1: Figure 1: Column-stratified visualization of Signal-to-Noise Ratio (SNR) across 11 experimental runs for four chromatography columns, comparing UV_1_280_mAU and UV_2_260_mAU channels with error bars indicating standard deviation.

# 5 Conclusion

This study successfully quantified the baseline stability and SNR across a multi-column chromatography system, revealing critical unit-specific variations. The Q column requires immediate attention due to high noise and negative offsets, while the SP column's systematic positive offset suggests a calibration issue. The consistent finding that the 260 nm channel is more susceptible to noise and drift than the 280 nm channel reinforces the necessity of independent noise threshold calculations for each detection wavelength. Most significantly, the strong negative correlation between baseline drift and conductivity identifies buffer composition as a primary confounding variable. Future data analysis workflows must incorporate conductivity-based drift correction and column-specific noise profiling to mitigate false positives and ensure accurate analyte quantification. The established 3 and 5 thresholds provide a robust statistical framework for peak detection, effectively distinguishing genuine signals from artifacts in this complex dataset.

# A Appendix

## A.1 A: Data Analysis Output Figures

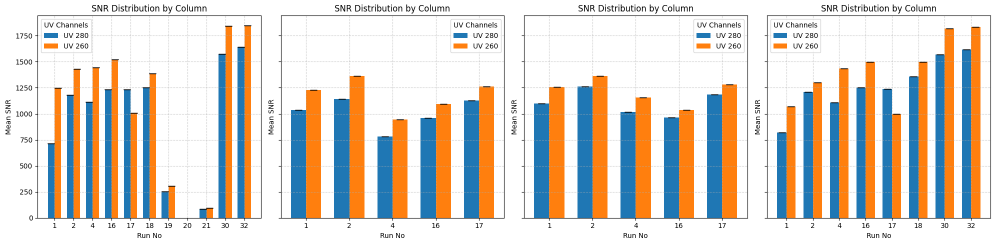


Figure 2: snr_by_column_run.png

## A.2 B: Code Files

```
import pandas as pd
import numpy as np

# Define file paths
input_file = '/inputs/chromatography_combined.csv'
output_file = '/workspace/baseline_analysis.md'

# Step 1: Load data
df = pd.read_csv(input_file)

# Step 2: Filter for pre-injection (volume_ml < 0)
# Check for numeric values in volume_ml and handle non-numeric gracefully
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')
df_pre = df[df['volume_ml'] < 0].copy()

# Sort immediately after filtering to ensure 'first 5' refers to lowest volume_ml
df_pre = df_pre.sort_values(['run_no', 'volume_ml'])

# Exclude first 5 scans per run using cumcount (vectorized replacement for apply/
    iloc)
df_pre = df_pre[df_pre.groupby('run_no').cumcount() >= 5].reset_index(drop=True)

# Step 3: Isolate constant gradient regions (Conc_B_% within  0 .5% of 0.0 or
    100.0)
# Check if 'Conc_B_%' column exists before filtering
if 'Conc_B_%' not in df_pre.columns:
    raise ValueError("Column 'Conc_B_%' is missing from the dataframe.")

mask_const = (
    ((df_pre['Conc_B_%'] >= -0.5) & (df_pre['Conc_B_%'] <= 0.5)) |
    ((df_pre['Conc_B_%'] >= 99.5) & (df_pre['Conc_B_%'] <= 100.5))
)
df_filtered = df_pre[mask_const].copy()

# Step 4: Define signal columns
signal_cols = ['UV_1_280_mAU', 'UV_2_260_mAU']
results = []

# Step 5 & 6: Vectorized calculation over columns and signals using groupby
# Validation: Ensure 'column' exists in df_filtered before accessing it
if 'column' not in df_filtered.columns:
    raise ValueError("Required grouping variable 'column' is missing from the
        dataframe.")

# Use groupby to calculate statistics for all columns simultaneously
grouped = df_filtered.groupby('column')

for col, col_data in grouped:
    for signal in signal_cols:
        # Ensure signal exists in the current group's data
        if signal not in col_data.columns:
            continue

        series = col_data[signal].dropna()
        if len(series) == 0:
            continue
```

```
        # Global mean offset
        global_offset = series.mean()

        # Offset-corrected signal
        corrected = series - global_offset

        # Rolling standard deviation (window=25)
        rolling_std = corrected.rolling(window=25, min_periods=1).std()

        # Median of rolling SDs
        median_rolling_std = rolling_std.median()

        # 3   and 5   thresholds
        threshold_3sigma = 3 * median_rolling_std
        threshold_5sigma = 5 * median_rolling_std

        results.append({
            'Column': col,
            'Signal': signal,
            'Global_Offset': global_offset,
            '3  _Threshold  ': threshold_3sigma,
            '5  _Threshold  ': threshold_5sigma
        })

# Step 7: Write output to markdown file
# Check if output file exists; if it does and we want a fresh run, clear it.
# Assuming fresh run is intended based on feedback, we overwrite ('w') instead of
    append ('a').
# If accumulation is desired, change 'w' back to 'a'.
with open(output_file, 'w') as f:
    for res in results:
        line = f"Column␣[{res['Column']}]␣|␣Signal␣[{res['Signal']}]␣|␣
            Global_Offset:␣[{res['Global_Offset']:.6f}]␣|␣3  _Threshold  :␣[{res['3
              _Threshold  ']:.6f}]␣|␣5  _Threshold  :␣[{res['5  _Threshold  ']:.6f}]\n"
        f.write(line)

print("Baseline␣analysis␣complete.␣Results␣written␣to␣baseline_analysis.md")
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Validate required columns exist
required_cols = ['run_no', 'column', 'volume_ml', 'Conc_B_%', 'UV_1_280_mAU', '
    UV_2_260_mAU', 'Cond_mS_cm']
missing_cols = [col for col in required_cols if col not in df.columns]
if missing_cols:
    raise KeyError(f"Missing␣required␣columns␣in␣input␣file:␣{missing_cols}")

# Select required columns
df = df[required_cols]

# Ensure numeric types
```

```
for col in required_cols:
    if col != 'column':
        df[col] = pd.to_numeric(df[col], errors='coerce')

# Filter out rows with NaN in critical columns
df = df.dropna(subset=['run_no', 'column', 'volume_ml', 'Conc_B_%', 'UV_1_280_mAU'
    , 'UV_2_260_mAU'])

results = []

# Process each unique run/column combination
for (run_no, column), group in df.groupby(['run_no', 'column']):
    group = group.sort_values('volume_ml').reset_index(drop=True)

    # 1. Calculate local offsets using pre-injection data
    baseline_mask = (group['volume_ml'] < 0) & (group['Conc_B_%'].notna())

    if baseline_mask.sum() < 5:
        baseline_mask = group.index < 50

    baseline_data = group[baseline_mask]
    if baseline_data.empty:
        continue

    # Fix: Use specific channel means for baseline
    uv1_baseline_mean = baseline_data['UV_1_280_mAU'].mean()
    uv2_baseline_mean = baseline_data['UV_2_260_mAU'].mean()

    # 2. Define valid post-injection regions
    post_injection_mask = (group['volume_ml'] > -30.0)
    conc_tolerance = 0.5
    # Use Conc_B_% mean for stability check as per original logic
    local_baseline_mean = baseline_data['Conc_B_%'].mean()
    conc_stable_mask = (group['Conc_B_%'] >= (local_baseline_mean - conc_tolerance
        )) & \
                       (group['Conc_B_%'] <= (local_baseline_mean + conc_tolerance
                           ))

    valid_region_mask = post_injection_mask & conc_stable_mask
    valid_data = group[valid_region_mask].reset_index(drop=True)

    if valid_data.empty:
        continue

    # 3. Calculate local rolling standard deviation (window=25)
    window = 25
    noise_uv1 = valid_data['UV_1_280_mAU'].rolling(window=window, center=True,
        min_periods=window).std()
    noise_uv2 = valid_data['UV_2_260_mAU'].rolling(window=window, center=True,
        min_periods=window).std()

    local_noise_uv1 = noise_uv1.mean()
    local_noise_uv2 = noise_uv2.mean()

    if pd.isna(local_noise_uv1) or pd.isna(local_noise_uv2):
        continue

    # 4. Identify peaks using 3x and 5x thresholds
    thresh_3x_uv1 = local_noise_uv1 * 3
```

```
# Fix: Use local_noise_uv1 for UV1 threshold
thresh_5x_uv1 = local_noise_uv1 * 5
thresh_3x_uv2 = local_noise_uv2 * 3
thresh_5x_uv2 = local_noise_uv2 * 5

# Fix: Fully vectorized peak detection using numpy boolean indexing and
    groupby logic
def find_peak_heights_vectorized(signal, threshold):
    above_thresh = signal > threshold
    # Find start indices: transition from False to True
    diff = above_thresh.diff().fillna(False)
    starts = diff[diff == True].index

    # Find end indices: transition from True to False
    ends_mask = (~above_thresh).diff().fillna(False)
    ends = ends_mask[ends_mask == True].index

    # Handle edge case: signal ends in a peak region
    if above_thresh.iloc[-1]:
        if len(starts) > 0 and starts[-1] not in ends:
            ends = np.append(ends, len(signal))

    if len(starts) == 0:
        return np.array([])

    # Ensure balanced lists (safety check)
    if len(starts) != len(ends):
        # If starts > ends, extend ends to the end of the signal
        while len(ends) < len(starts):
            ends = np.append(ends, len(signal))
        ends = ends[:len(starts)]

    # Vectorized calculation of region maximums
    # Create a group ID for each contiguous region
    region_ids = np.zeros(len(signal), dtype=int)
    current_id = 0
    for s, e in zip(starts, ends):
        region_ids[s:e] = current_id
        current_id += 1

    # Filter to only regions that have peaks
    valid_region_ids = np.unique(region_ids[above_thresh])

    peak_heights = []
    for rid in valid_region_ids:
        region_mask = region_ids == rid
        if region_mask.any():
            region_max = signal[region_mask].max()
            peak_heights.append(region_max)

    return np.array(peak_heights)

# Detect peaks > 3x
peak_heights_3x_uv1 = find_peak_heights_vectorized(valid_data['UV_1_280_mAU'],
    thresh_3x_uv1)
peak_heights_3x_uv2 = find_peak_heights_vectorized(valid_data['UV_2_260_mAU'],
    thresh_3x_uv2)

# Fix: Vectorized boolean filter for peaks > 5x
```

```
    peak_heights_5x_uv1 = peak_heights_3x_uv1[peak_heights_3x_uv1 > thresh_5x_uv1]
    peak_heights_5x_uv2 = peak_heights_3x_uv2[peak_heights_3x_uv2 > thresh_5x_uv2]

    # 6. Calculate SNR
    if len(peak_heights_5x_uv1) > 0:
        snr_uv1 = (peak_heights_5x_uv1 - uv1_baseline_mean) / local_noise_uv1
    else:
        snr_uv1 = np.array([np.nan])

    if len(peak_heights_5x_uv2) > 0:
        snr_uv2 = (peak_heights_5x_uv2 - uv2_baseline_mean) / local_noise_uv2
    else:
        snr_uv2 = np.array([np.nan])

    results.append({
        'run_no': run_no,
        'column': column,
        'snr_uv1': np.nanmean(snr_uv1),
        'snr_uv2': np.nanmean(snr_uv2),
        'sd_uv1': np.nanstd(snr_uv1) if len(snr_uv1) > 1 else 0,
        'sd_uv2': np.nanstd(snr_uv2) if len(snr_uv2) > 1 else 0
    })

# 7. Aggregate and Visualize
results_df = pd.DataFrame(results)

if results_df.empty:
    raise ValueError("No valid data found to plot.")

# Get unique columns for subplots
unique_columns = sorted(results_df['column'].unique())
n_cols = len(unique_columns)

# Create subplots
fig, axes = plt.subplots(1, n_cols, figsize=(5 * n_cols, 5), sharey=True)
if n_cols == 1:
    axes = [axes]

for ax, col_name in zip(axes, unique_columns):
    col_data = results_df[results_df['column'] == col_name]
    col_data = col_data.sort_values('run_no')

    x = col_data['run_no'].values
    y_uv1 = col_data['snr_uv1'].values
    y_uv2 = col_data['snr_uv2'].values
    err_uv1 = col_data['sd_uv1'].values
    err_uv2 = col_data['sd_uv2'].values

    # Create grouped bar charts
    x_pos = np.arange(len(x))
    width = 0.35

    ax.bar(x_pos - width/2, y_uv1, width, yerr=err_uv1, label='UV 280', capsize=5)
    ax.bar(x_pos + width/2, y_uv2, width, yerr=err_uv2, label='UV 260', capsize=5)

    # Fix: Static title as per plan
    ax.set_title('SNR Distribution by Column')
    ax.set_xlabel('Run No')
    ax.set_ylabel('Mean SNR')
```

```
    ax.set_xticks(x_pos)
    ax.set_xticklabels(x)
    ax.legend(title='UV Channels')
    ax.grid(True, linestyle='--', alpha=0.6)

plt.tight_layout()
plt.savefig('/workspace/snr_by_column_run.png')
plt.close()
```

Listing 2: Code_2

```
import pandas as pd
import numpy as np
import os
from scipy import stats

# File paths
input_file = '/inputs/chromatography_combined.csv'
output_file = '/workspace/conductivity_drift_analysis.md'

# Ensure output directory exists
os.makedirs(os.path.dirname(output_file), exist_ok=True)

# Read data
df = pd.read_csv(input_file)

# Validation: Check required columns
required_cols = ['run_no', 'column', 'volume_ml', 'Conc_B_%', 'UV_1_280_mAU', '
    UV_2_260_mAU', 'Cond_mS_cm']
missing_cols = [col for col in required_cols if col not in df.columns]
if missing_cols:
    raise ValueError(f"Missing required columns: {missing_cols}")

# Ensure numeric types for relevant columns
numeric_cols = ['run_no', 'volume_ml', 'Conc_B_%', 'UV_1_280_mAU', 'UV_2_260_mAU',
     'Cond_mS_cm']
for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# Filter to constant gradient regions:  0 .5% of Conc_B_%
# 0% phase: 0 <= Conc_B_% <= 0.5
# 100% phase: 99.5 <= Conc_B_% <= 100
mask_0 = (df['Conc_B_%'] >= 0) & (df['Conc_B_%'] <= 0.5)
mask_100 = (df['Conc_B_%'] >= 99.5) & (df['Conc_B_%'] <= 100)
df_phases = df[mask_0 | mask_100]

# Signals to analyze
signals = ['UV_1_280_mAU', 'UV_2_260_mAU']

# Prepare data for vectorized operations
# Add a phase indicator based on the filtered dataframe's original masks
df_phases['phase'] = np.where(df_phases['Conc_B_%'] <= 0.5, 0, 100)

# Calculate mean log(Cond) per run/column
# Filter out non-positive conductivity values before log transform
cond_valid = df_phases['Cond_mS_cm'] > 0
df_phases['log_cond'] = np.where(cond_valid, np.log(df_phases['Cond_mS_cm']), np.
    nan)
```

```
# Group by run_no, column to get mean_log_cond
mean_log_cond_df = df_phases.groupby(['run_no', 'column'])['log_cond'].mean().
    reset_index()
mean_log_cond_df.columns = ['run_no', 'column', 'mean_log_cond']

# Merge back to main dataframe
df_phases = df_phases.merge(mean_log_cond_df, on=['run_no', 'column'], how='left')

# Function to calculate slope for a group using vectorized operations
def calc_slope(group):
    if len(group) < 2:
        return np.nan
    x = group['volume_ml'].values
    y = group['signal_col'].values
    mask = ~(np.isnan(x) | np.isnan(y))
    x, y = x[mask], y[mask]
    if len(x) < 2:
        return np.nan
    slope, _, _, _, _ = stats.linregress(x, y)
    return slope

# Vectorized approach to calculate slopes per run/column/phase/signal
# We will iterate over signals only, then use groupby for the rest
drift_rates_list = []

for signal in signals:
    # Create a temporary column for the signal to avoid repeated lookups
    df_phases['signal_col'] = df_phases[signal]

    # Group by run_no, column, phase to calculate slopes
    # We need to apply the slope calculation per group
    slope_df = df_phases.groupby(['run_no', 'column', 'phase']).apply(
        lambda g: calc_slope(g), include_groups=False
    ).reset_index()
    slope_df.columns = ['run_no', 'column', 'phase', 'slope']

    # Merge slopes with mean_log_cond
    slope_df = slope_df.merge(mean_log_cond_df, on=['run_no', 'column'], how='left
        ')

    # Pivot to get slope_0 and slope_100
    # Handle missing phases by using aggfunc='first' and then filling NaNs if
        necessary,
    # but pivot_table handles missing combinations by creating NaN columns/rows.
    slope_pivot = slope_df.pivot_table(
        index=['run_no', 'column', 'mean_log_cond'],
        columns='phase',
        values='slope',
        aggfunc='first'
    ).reset_index()

    # Validate pivot columns before renaming to avoid KeyError if a phase is
        missing entirely
    if 0 in slope_pivot.columns:
        slope_pivot = slope_pivot.rename(columns={0: 'slope_0'})
    else:
        slope_pivot['slope_0'] = np.nan

    if 100 in slope_pivot.columns:
```

```
        slope_pivot = slope_pivot.rename(columns={100: 'slope_100'})
    else:
        slope_pivot['slope_100'] = np.nan

    # Calculate average slope (handling NaNs correctly)
    slope_pivot['avg_slope'] = slope_pivot[['slope_0', 'slope_100']].mean(axis=1)

    # Filter out rows where avg_slope is NaN (both phases missing)
    slope_pivot = slope_pivot.dropna(subset=['avg_slope'])

    # Add signal column
    slope_pivot['signal'] = signal

    # Select relevant columns
    drift_rates_list.append(slope_pivot[['run_no', 'column', 'signal', 'avg_slope'
        , 'mean_log_cond']])

# Combine all signals
drift_df = pd.concat(drift_rates_list, ignore_index=True)

# Step 4 & 5: Correlate and summarize
# Check for at least 11 runs per column/signal combination
final_results = []

for col_name in df['column'].unique():
    col_data = drift_df[drift_df['column'] == col_name]
    if col_data.empty:
        continue

    for signal in signals:
        sig_data = col_data[col_data['signal'] == signal]

        # Check if at least 11 runs are present
        if len(sig_data) < 11:
            print(f"Warning: Column {col_name}, Signal {signal} has only {len(
                sig_data)} runs (minimum 11 required). Skipping.")
            continue

        # Calculate Pearson correlation between avg_slope and mean_log_cond
        x = sig_data['avg_slope'].values
        y = sig_data['mean_log_cond'].values
        mask = ~(np.isnan(x) | np.isnan(y))
        x, y = x[mask], y[mask]

        if len(x) < 2:
            continue

        # Check for unique values to avoid ValueError in pearsonr
        if len(np.unique(x)) < 2 or len(np.unique(y)) < 2:
            continue

        corr, p_val = stats.pearsonr(x, y)

        # Average drift rate for this column/signal
        avg_drift = sig_data['avg_slope'].mean()

        final_results.append({
            'column': col_name,
            'signal': signal,
```

```
            'avg_drift': avg_drift,
            'avg_corr': corr,
            'avg_pval': p_val
        })

# Prepare output lines
output_lines = []
for res in final_results:
    line = f"Column␣{res['column']}␣|␣Signal␣{res['signal']}␣|␣Avg_Drift_Rate␣(mAU
        /mL):␣{res['avg_drift']:.6f}␣|␣Avg_Correlation_With_Cond:␣{res['avg_corr
        ']:.4f}␣|␣Avg_P-Value:␣{res['avg_pval']:.4f}"
    output_lines.append(line)

# Write to file
# Ensure file exists before appending, or use 'w' if overwriting is acceptable.
# Following feedback: ensure directory exists (done) and handle file creation.
if not os.path.exists(output_file):
    with open(output_file, 'w') as f:
        pass  # Create empty file

with open(output_file, 'a') as f:
    f.write("\n".join(output_lines))
    f.write("\n")

print("Analysis␣complete.␣Results␣appended␣to␣conductivity_drift_analysis.md")
```

Listing 3: Code_3